A New Golden Age for Computer Architecture

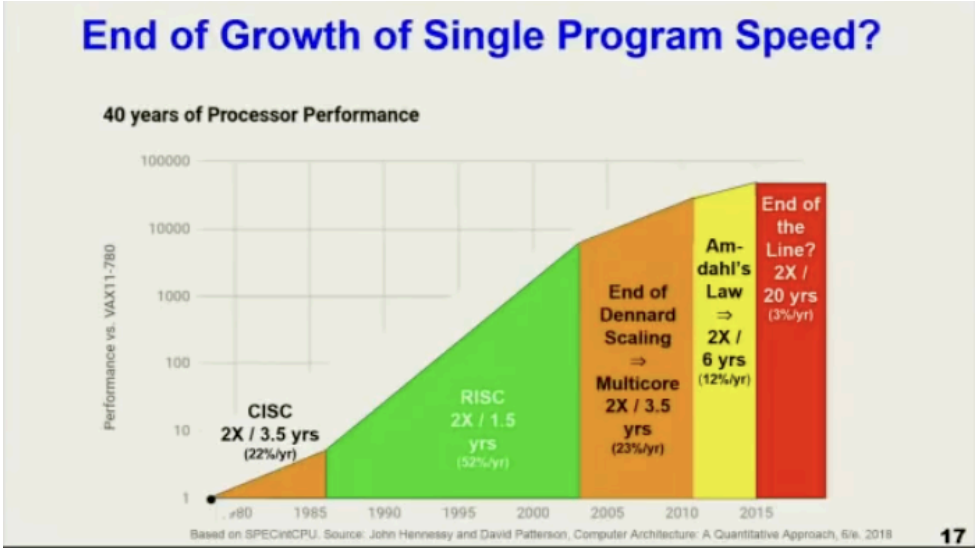
—----------History—----------

Instruction Set Architecture:

- Software talks to hardware using a vocabulary
    - Words are called instructions
    - Vocabulary called instruction set architecture (ISA)
    - A program is a book, written in words of the vocabulary
    - ISA is like a manual: “If you send the binary pattern 0001, my chip will perform an addition”
- IBM Compatibility Problem in Early 1960s
    - IBM had 4 incompatible lines of computers, and people wonder if they can unify the lines
    - This unified ISA is called the IBM System/360
- Control vs Datapath
    - Each processor is split into 2 functional areas
        - Datapath: where the actual math happens
            - Contains registers (storage units), ALU (Arithmetic Logic Unit for calculating), and wires that move numbers around
        - Control: the “brain” that tells the datapath what to do and in what order
    - Designing the Control unit was difficult because it is complex
    - Maurice Wikes invented the idea of microprogramming to design the control unit of a processor
        - Microprogramming breaks macro-instructions into simple instructions. The recipe of breaking down the instructions are stored in a ROM (Read Only Memory Unit).
        - Hence, if you want to modify a macro-instruction, you don’t need to rewire all the electronic gates, you just need to swap a new ROM with the new instructions.
- IC Technology, Microcode, and CISC
    - With Moore’s law, memory for control store could grow
    - Now, semiconductor ram have almost the same speed as ROM
    - With RAM (you can rewrite stuff in there), it is easier to fix microcode bugs
    - This allowed for more complicated ISAs (CISC, for example)
- Microprocessor Evolution
    - Rapid progress in the 1970s, fueled by advances in MOS technology, shrunk logic inside giants cabinets into a single silicon chip—the microprocessor
    - “The Microprocessor Wars”: companies compete by cramming more instructions
    - Intel iAPX 432: ambitious, but late
    - Motorola 68000 was late
    - Intel 8086 was a stopgap, but because it was picked by the IBm PC, it became an overnight success
- Analyzing Microcoded Machines 1980s

    - Hardware & software interface rises from assembly to high-level programming
    - Unix is the first operating system written in high-level language (which is C)
    - In the Assembly Language age, since humans are writing instructions directly, hardware designers tried to make those instructions “rich” and “complex” to make the human’s job easier
    - Performance is measured by how many instructions you can pack into a program
    - As programming moved to high-level languages, people started using compilers
    - Measuring performance now meant looking at the code the compilers actually produced
- From CISC to RISC (reduced instruction set computer)
    - Instructions as simple as microinstructions, but not as wide
    - Enable pipelined implementations (multiple instructions are in different stages of completion at the same time)
    - The Cache: CPU fetches a whole bunch of instructions from the Main memory & stores them in RAM (the Cache), allowing for faster execution of instructions.
    - Chaitin’s register allocation scheme benefits load-store ISAs
- Berkeley and Stanford RISC Chips
- “Iron Law” of Processor Performance: How RISC can win
    - RISC executes more instructions/program (25% more instructions) but many fewer clock cycles per instruction (around ⅕ CPI)
    - It’s 4 times faster than CISC
- $$\frac{\text{Time}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} * \frac{\text{Clock cycles}}{\text{Instruction}} * \frac{\text{Time}}{\text{Clock cycle}}$$
- CISC vs RISC Today
    - PC Era
    - Modern x86 chips (Intel & AMD) translate x86 instructions into internal RISC instructions
    - By doing this, the chip can keep the “rich” vocabulary that makes software compatible (CISC) while using high-speed assembly-line techniques (RISC) to actually do the work
    - MPU vs. IP in SoC
        - MPU (microprocessor unit): the brain, which should be plugged into the motherboard and communicates with other chips (GPU, RAM, Wi-Fi controller etc.) through physical wires on the motherboard. Causes latency and demands a lot of battery power.
        - SoC: everything(the CPU, graphics, etc.) is moved onto a tiny square of silicon
            - There are specialized IP blocks: pre-designed blueprints for specific tasks like video, security, npu
            - Efficiency matters as much as performance (value die area, energy as much as performance)
        - 99% of the processors are RISC

—----------Challenges—----------

- Moore's law is slowing down
- Dennard Scaling: how can we put so many transistors on the chip & have the chip not burn up? More transistors w/ lower threshold voltage=>same power usage
    - Around 2008, we can no longer lower the threshold voltage so power budget went up
    - 

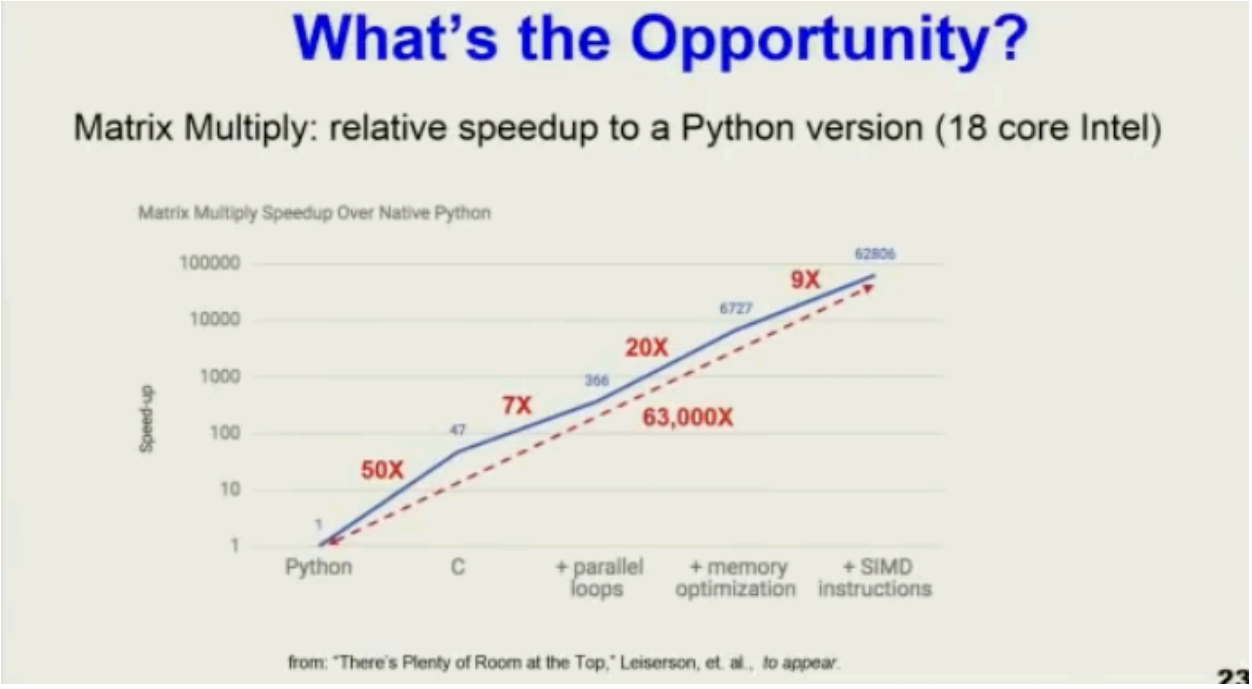
- Quantum Computing to the Rescue?
    - For Cloud, not Client
    - Without a "killer app" or at leat a useful app runnable in the first 10 years, progress may stall
    - Significant technical and financial issues remain towards building a large, fault-tolerant quantum computer and one is unlikely to be built within the coming decade (Gwryrine, Peter (2019))
- Current Security Challenge
    - Spectre Execution, modern CPUs are impatient: if they see an "if" statement, they don't wait for an answer. They guess the result and start running the code immediately.
    - If the CPU guesses wrong, it eventually realizes its mistake, "rewinds" the clock and deletes the work.
    - However, micro-architecturally, a trace is left behind
        - The CPU might have loaded a piece of data from memory in the the Cache
        - Even though the CPU "deleted" the result, it forgot to clear the Cache.
    - If a hacker runs their code and asks for the same piece of data
        - If they get it in 10 ns, they know it's already in the Cache (the CPU's wrong guess accessed that specific piece of data)
        - If it takes 200 ns, the data was not in the cache
        - Need Computer Architecture 2.0 to fix the security leaks

—----------Opportunities—----------

- Software-centric:
    - Modern scripting languages are interpreted, dynamically-typed and encourage resume
    - This is efficient for programmers but not for execution
- Hardware-centric:
    - The only path left is domain-specific architectures
    - Just do a few tasks, but extremely well
- COmbination:
    - Domain specific languages & architectures

What's the Opportunity:

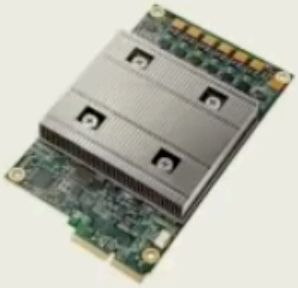
- For matrix multiply, going from python => C is 50x faster!

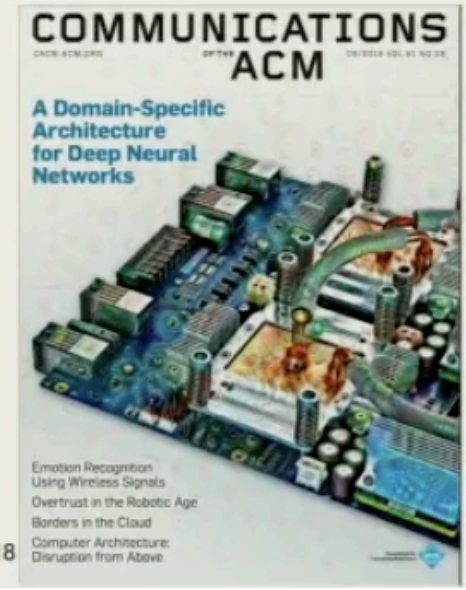


- Why DSAs (Domain Specific Architecture) can Win
    - More effective parallelism for a specific domain
    - More effective use of memory bandwidth
    - Eliminate unneeded accuracy
    - Domain specific programming language provides with for software
- Deep learning is causing a machine learning revolution
    - Tensor processing units v1

Tensor Processing Unit v1 (Announced May 2016)

Google-designed chip for neural net **inference**



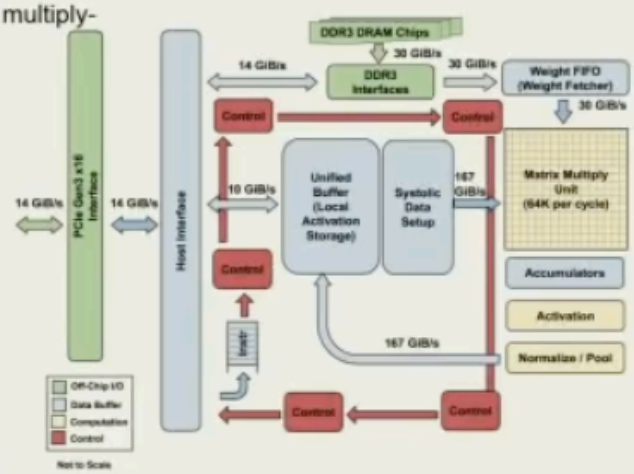


In production use for 4 years: used by billions on search queries, for neural machine translation, for AlphaGo match, …

A Domain-Specific Architecture for Deep Neural Networks, Jouppi, Young, Patil, Patterson, *Communications of the ACM*, September 2018

TPU: High-level Chip Architecture

- The Matrix Unit: 65,536 (256x256) 8-bit multiply-accumulate units
- 700 MHz clock rate
- Peak: 92T operations/second
  - 65,536 * 2 * 700M
- >25X as many MACs vs GPU
- >100X as many MACs vs CPU
- 4 MiB of on-chip Accumulator memory + 24 MiB of on-chip Unified Buffer (activation memory)
- 3.5X as much on-chip memory vs GPU
- 8 GiB of off-chip weight DRAM memory

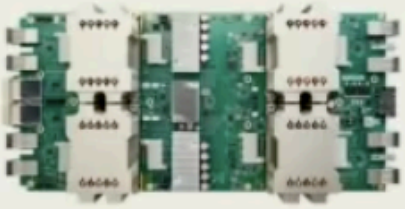


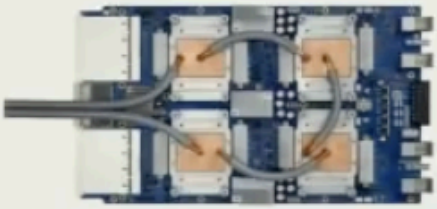
ML Training Trends

Since 2012,
AI training state of the art compute demand 10X per year!
(Moore's Law "only" 10X in 5 years)

From "AI and Compute." Dario Amodei and Danny Hernandez, May 16, 2018

Training: TPUv2 (5/2017)TPUv3 (5/2018)





Peak: 11.5 PetaFLOP/s
(~top 25 supercomputer)

Peak: >100 PetaFLOP/s
(~top 5 supercomputer)

Current Neural Network Architecture Debate

- Google TPU: 1 core per chip, large 2D multiplier, software controlled memory (instead of caches)
- NVIDIA GPU: 80 cores, many threads (20MB registers), small multipliers, caches, scatter/gather & coalescing HW
- Microsoft FPGA: customize "hardware" to application
- Intel CPU: 30+ cores, 3 levels of caches, SIMD instructions
  - Also bought Altera that supplies Microsoft's FPGAs
  - Also bought Nervana, Movidius, MobilEye to offer custom chip DSA
- > 100 startups with their own architecture bets
- *#3. Ultimately the marketplace settles architecture debates*

34

Why Instruction Set Architecture matters

- **Why can't Intel sell mobile chips?**
  - 99%+ of mobile phones/tablets based on ARM v7/v8 ISA
- **Why can't ARM partners sell servers?**
  - 99%+ of laptops/desktops/servers based on x86 ISA
- **How can IBM still sell mainframes?**
  - IBM 360, oldest surviving ISA (50+ years)

ISA is most important interface in computer system

where software meets hardware

- Companies and ISAs can come and go
- RISC-V: open & free ISA (wow that's very cool)

OURS (睿思芯科) energy-efficient RISC-V AI Chip for IoT

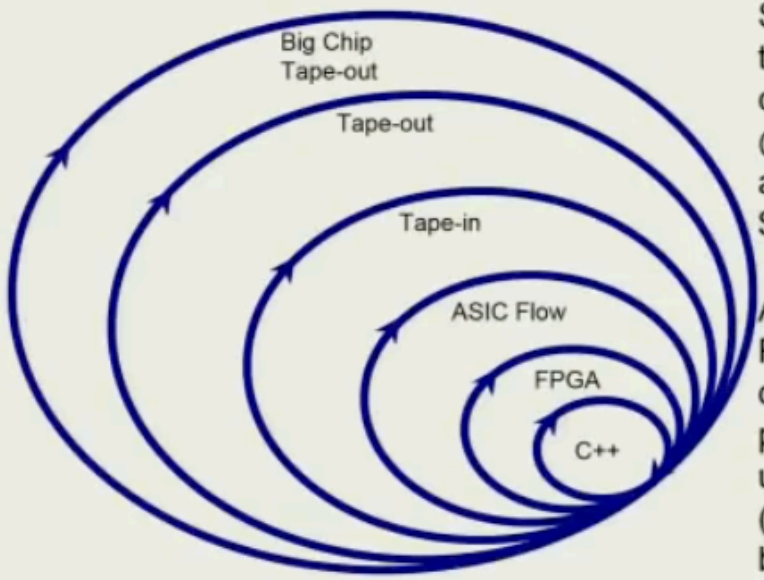
OURS Pygmy microprocessor

- 28nm HPC+ TSMC @ 600 MHz
  - From scratch to tapeout ~7 months
  - (Thanks to the RISC-V infrastructure)
- Full RISC-V based heterogenous multicore architecture
  - 64-bit control processor (RV64g)
    - ~ 10mW active
  - 12 energy-efficient AI engines based on custom RV vector extensions
    - INT8 : ~4 TOPS/watt
    - FP16 : ~0.35 TOPS/watt
- 1MB SRAM, LPDDR4 support
- Retail price < $3

Security and Open Architecture

- Security community likes simple, verifiable (no trap doors), alterable, free and open architecture and implementations
- Equally important is number of people and organizations performing architecture experiments
  - Want all the best minds to work on security
- Plasticity of FPGAs + open source RISC-V implementations and SW ⇒ novel architectures can be deployed online, subjected to real attacks, evaluated & iterated in weeks vs years (even 100 MHz OK)

Agile Hardware Development Methodology



Lee, Y., Waterman, A., Cook, H., Zimmer, B., Keller, B., Puggelli, A., ... & Chiu, P. F. (2016). "An agile approach to building RISC-V microprocessors." *IEEE Micro*, *36*(2), 8-20.

Small chip tape-out 100 chips 1x1mm @ 28nm is affordable at $14,000!

AWS FPGA F1 instance ⇒ develop new prototypes using cloud (nothing to buy)

Lessons of last 50 years of Computer Architecture

1. *Software advances can inspire architecture innovations*
   - Microprogramming - control as software
   - RISC, x86 ISA - (Hardware) translator vs interpreter
   - Open Architectures and Implementations
   - Agile Hardware Development
2. *Raising the HW/SW interface enables arch. opportunities*
   - Assembly to HLL ⇒ RISC
   - HLL to Domain Specific Language ⇒ DSA
3. *Ultimately the marketplace settles architecture debates*
   - Losers: 432
   - Winners: IBM S/360, 8086 (PC Era), RISC (Post PC Era)
   - Open vs Proprietary ISA (RISC-V vs ARM): Too soon to tell
   - ML DSA (SIMD vs GPU vs TPU vs FPGA vs startups): Too soon to tell